

Développeur 1 : Spécialiste Vue (Front-End & UI)

Son rôle : Il crée l'interface d'interaction et la structure visuelle de l'export.

- **Fichiers de travail :** templates/page-admin.php & assets/css/style.css
- **Tâches précises :**
 1. Écrire le code HTML pur du panneau d'exportation avec les boutons et options de sélection.
 2. Intégrer la charte graphique SolDigit (mode sombre avec accents dorés) pour que l'interface soit soignée.
 3. Intégrer une structure HTML/CSS pour la **barre de progression** (un conteneur gris et une jauge interne dorée qui évoluera dynamiquement).

Développeur 2 : Expert JavaScript & Intégration AJAX (Liaison Vue-Contrôleur)

Son rôle : Il donne vie à l'interface en pilotant la boucle asynchrone depuis le navigateur.

- **Fichier de travail :** assets/js/script.js
- **Tâches précises :**
 1. Écouter l'événement du clic sur le bouton "Exporter".
 2. Coder la fonction récursive/boucle lancer_chunk_export(index).
 3. Réceptionner la réponse JSON du serveur : si le statut est processing, il calcule le pourcentage d'avancement, demande au Développeur 1 de faire avancer la jauge dorée, puis relance immédiatement la requête avec le nouvel index. Si c'est completed, il affiche le message de succès.

Développeur 3 : Concepteur du Contrôleur AJAX (L'Aiguilleur)

Son rôle : Il est le point de contact sur le serveur. Il valide la sécurité et appelle le moteur.

- **Fichier de travail :** plugin-migration.php (Section Hooks & Callbacks)
- **Tâches précises :**
 1. Enregistrer la route AJAX auprès de WordPress via le hook add_action('wp_ajax_...', ...).
 2. Sécuriser l'accès : vérifier que l'utilisateur possède les droits d'administration (manage_options).
 3. Extraire les variables de la requête (\$_POST['index']), instancier la classe Exportation du Modèle, lui passer les paramètres, puis formater et renvoyer la réponse JSON standardisée (wp_send_json_success).

Développeur 4 : Développeur du Modèle - Algorithme ZIP (Le Cœur Métier)

Son rôle : Il implémente la logique lourde de découpage et de compression des fichiers.

- **Fichier de travail :** includes/class-exportation.php
- **Tâches précises :**
 1. Coder la méthode de scan récursif (collecter_tous_les_fichiers) pour lister le contenu de wp-content en prenant soin d'exclure le dossier de destination des sauvegardes.

2. Implémenter la méthode principale `zipper_paquet_fichiers($nom_zip, $index_depart)`. Elle doit extraire un sous-ensemble (quota de 100 fichiers), ouvrir le fichier ZIP de la session, y ajouter ces fichiers, sauvegarder et retourner le prochain index (ou 'termine').

Développeur 5 : Développeur du Modèle - Suivi & Sécurité (Anti-Plantage)

Son rôle : Il gère la persistance de l'état de l'export et s'assure que le serveur ne sature pas.

- **Fichiers de travail :** `includes/class-backup.php` & `includes/class-exportation.php` (Section Sécurité)
- **Tâches précises :**
 1. Coder dans `class-backup.php` la création du ticket en BDD lors de la toute première requête (statut `pending`) et la mise à jour à chaque étape (statut `processing_files`, puis `completed` ou `failed`).
 2. Écrire la fonction de garde-fou (`surveiller_serveur`) dans le modèle d'export : mesurer la mémoire vive consommée par l'algorithme du Développeur 4. Si la mémoire dépasse 85% de la configuration serveur, il lève une exception, force le statut de la sauvegarde à `failed` en BDD et prépare le message d'échec pour le Contrôleur.

Plan de synchronisation pour l'équipe (À faire avant de coder) :

Pour que l'équipe fusionne son code demain sans aucun bug, réunissez-vous 5 minutes pour figer le **contrat d'interface** (le format du JSON) :

```
JSON
// Format de retour attendu (JS) et envoyé par (Contrôleur)
{
  "success" true
  "data"
    "statut" "processing"
    "prochain_index" 100
    "total_fichiers" 1250
}
```